

The Transported Probability Density Function (PDF) Method and its Relation to the Black-Scholes-Merton Equation

Christian DiPietrantonio

ME 5311: Computational Methods to Viscous Flows
University of Connecticut
May 1, 2026

Contents

1	Introduction	1
2	Financial Derivatives and the Black-Scholes-Merton Equation	1
2.1	Financial Derivatives and Options	1
2.2	The Black-Scholes-Merton Equation	2
3	Derivation of the Black-Scholes-Merton Equation	3
3.1	Stochastic Calculus and Itô's Lemma	3
3.2	The Stock Price Model: Geometric Brownian Motion	4
3.3	Applying Itô's Lemma to the Option Value	4
3.4	Constructing the Hedged Portfolio	4
3.5	No Arbitrage Condition	5
4	The Transported PDF Method	5
4.1	Statistical Description of Turbulent Flows	5
4.2	Eulerian PDF Transport Equation	5
4.3	The Generalized Langevin Model	6
4.4	Derivation of the PDF Transport Equation	6
5	The Connection Between the BSM Equation and the Transported PDF Method	6
5.1	Structural Equivalence of the SDEs	6
5.2	Variable Mapping	7
6	Solution Methods	8
6.1	Eulerian Methods	8
6.2	Lagrangian Methods	8
6.3	Constant vs. Variable Coefficients	8
7	Computational Experiments and Results	8
7.1	Problem Description	8
7.2	Analytical Solution	8
7.3	Finite Difference Solver (Eulerian)	8
7.4	Monte Carlo Solver (Lagrangian)	8
7.5	Results: Option Pricing	9
7.6	Results: Monte Carlo Visualization and PDF Transport	9
7.7	Results: PDF Transport Continued	9
7.8	Results: Monte Carlo Convergence	10
8	Conclusions	10
A	Copy of Program Listing	12

1 Introduction

This paper explores the mathematical connection between the Black-Scholes-Merton (BSM) equation for pricing financial derivatives and the transported probability density function (PDF) method employed in computational fluid dynamics (CFD). The central argument is that these two equations, from completely different fields, are derived from stochastic differential equations (SDEs) of the same drift plus diffusion form, and can be solved using the same classes of numerical methods. The BSM equation is derived from the geometric brownian motion SDE, while the transported PDF equation is derived from the generalized Langevin SDE for the velocity of a fluid particle in turbulence. Even though the two equations were developed in separate disciplines, these two equations share a mathematical connection through stochastic calculus.

The paper begins by establishing the financial definitions and background necessary to the understanding of the further sections. Following this, the BSM equation is derived following the approach outlined by Hull [1] and Merton [2]. The transported PDF method is then introduced following the work of Pope [3], and the connection between it and the BSM equation is illustrated through comparison of the underlying SDEs and variable mapping. The paper then outlines the solution methods for the BSM and the transported PDF equation, which include grid-based Eulerian and particle-based Lagrangian approaches. To demonstrate these solution methods, a Python program was developed that implements three different solution methods for pricing a European call option. The first solution method is simply the analytical solution to the BSM equation. Following this, the solver implements an implicit finite difference scheme (Eulerian approach), and a Monte Carlo simulation (Lagrangian approach). The results are compared to one another, and also used to visualize the transport of the probability density function of the stock price. The latter of which can be directly related to the transported PDF of the velocity of a fluid particle as obtained by the transported PDF method. The benefits and pitfalls of the different solution approaches are also explored. Finally, the paper concludes by discussing the real-world applications of this cross-disciplinary connection, and how advancements in either quantitative finance or computational fluid dynamics have potential to be utilized by the other.

To begin the comparison of the equations and methods, a foundational understanding of the Black-Scholes-Merton equation is necessary. [1]

2 Financial Derivatives and the Black-Scholes-Merton Equation

2.1 Financial Derivatives and Options

In order to understand the Black-Scholes-Merton equation, a foundational understanding of financial derivatives, specifically options, is required. According to Hull, a derivative is a “financial instrument whose value depends on (or derives from) the values of other, more basic, underlying variables” [1]. A stock option, for example, is a derivative whose value is dependant on the price of a stock. However, different derivatives can be dependant on almost any variable, even the weather. For this paper, the primary derivatives of interest are

options, which are securities that give “the right to buy or sell an asset, subject to certain conditions, within a specified period of time” [4].

The two major types of options are call and put options. A call option gives the holder the right, but not the obligation, to buy the underlying asset by a certain data for a certain price. A put option gives the holder the right, but not the obligation, to sell the underlying asset by a certain data for a certain price [1]. Options can also be further classified by when they can be excercised. American options can be excercised at any time up to the expiration date as specified in the contract, while European options can only be excercised on the expiration date itself [4]. The price at which the asset may be bought or sold is refered to as the strike price, K , and the expiration date can an also be refrered to as the maturity date, T .

For the remainder of this paper, the focus will be on European call options. The value of a European call option at expiry can be determined by the following payoff function:

$$V = \max(S_T - K) \quad (1)$$

Where S_T is the stock price at expiry. From this payoff function one can observe that, if the stock price exceeds the strike price at expiry, the holder profits. However, if the stock price at expiry is below the strike price, the option expires worthless, as the holder is not obligated to execute the option. In this latter scenario, the holder only loses what they paid for the option. The Black-Scholes-Merton equation aims to determine a price for the contract, before its value is known at expiry.

2.2 The Black-Scholes-Merton Equation

In deriving their valuation formula, Black and Scholes [4] assume an idealized, frictionless market with continuous trading, no transaction costs, constant interest rates, no dividends, unrestricted short selling, and the ability to borrow or lend at the risk-free rate. The option is also assumed to be a European call option. Furthermore, an arbitrage-free market is also assumed, which means that no trading strategy can generate riskless profits. More explicitly, an arbitrage opportunity can be formally defined as “a (self-financing) trading strategy” that starts with zero value and terminates with a positive value. Therefore, a market is arbitrage-free if there are no such arbitrage opportunities [5].

The assumptions outlined above lead to the derivation of the following second-order linear parabolic Partial Differential Equation (PDE) for the price $V(S, t)$ of a European call option. The same equation was independently derived by Robert C. Merton [2] through the use of stochastic calculus, and therefore is commonly referred to as the Black-Scholes-Merton PDE:

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (2)$$

Where S is the underlying stock price, t is time, r is the risk-free interest rate, and σ is the volatility of the stock, a measure of how much the stock price fluctuates, or in other words a “measure of our uncertainty about the returns provided by the stock” [1]. This equation was obtained by forming a hedged portfolio that eliminates risk and requiring that its returns equal the risk-free rate in an arbitrage-free market.

It should also be noted that the structure of Equation 2 can be recognized as having the same form of the general transport equation given the context of this course. Where $\frac{\partial V}{\partial t}$ is the temporal term, $\frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2}$ is a diffusion term, $rS \frac{\partial V}{\partial S}$ is an advection term, and rV is a source (in this case, decay) term. This course has dedicated ample time to discretizing and computationally solving equations of this exact form.

3 Derivation of the Black-Scholes-Merton Equation

The use of Stochastic Calculus in the derivation of Equation 2 is crucial to the argument of this paper. Therefore, this section will walk through such derivation as done so by Hull [1] and Merton [2].

3.1 Stochastic Calculus and Itô's Lemma

GO TO SHREVE FOR DEFINITION OF STOCHASTIC CALCULUS

Before completing the derivation, it is important to highlight a few concepts from Stochastic Calculus that are used in the derivation. First, it should be noted that Stochastic Calculus is a branch of mathematics built specifically to handle the derivatives and integrals of stochastic processes, which are by definition, “an equation describing the probabilistic behavior of a stochastic variable” [1]. Furthermore, a stochastic variable can be defined as a variable “whose future is uncertain”, or in other words, random. The first process to consider is the generalized Wiener process:

$$dx = a dt + b dz \quad (3)$$

Where $dz = \epsilon \sqrt{\Delta t}$ is referred to as a Wiener process, and ϵ has a standard normal distribution. Further, an Itô process is a generalized Wiener process where the parameters a and b are functions of the underlying variable and time, and can be expressed as:

$$dx = a(x, t) dt + b(x, t) dz \quad (4)$$

In standard calculus, the chain rule can be used to find the derivative or differential of a multivariable function. However, in Stochastic Calculus, one must use Itô's Lemma to compute the stochastic differential of a function that depends on a stochastic process. Doing so accounts for the fact that second-order terms can not be ignored in the formulation of the differential. This is because the diffusion term in a stochastic process (at least the ones within the scope of this paper) depends on a Wiener Process $dz \sim \sqrt{\Delta t}$, and when squared, this term is on the order of dt . Itô's lemma shows that for a function $G(x, t)$, where x is a stochastic variable following an Itô process, the differential of G can be expressed as

$$dG = \frac{\partial G}{\partial x} dx + \frac{\partial G}{\partial t} dt + \frac{1}{2} \frac{\partial^2 G}{\partial x^2} b^2 dt \quad (5)$$

Again, the key difference from ordinary calculus is the presence of the second-order term $\frac{\partial^2 G}{\partial x^2} b^2$ [1].

3.2 The Stock Price Model: Geometric Brownian Motion

Returning to the topics of this paper, the price of a stock S is modeled as a stochastic process using the following stochastic differential equation:

$$dS = rS dt + \sigma S dW \quad (6)$$

where r is the stocks expected rate of return (in this case, the risk-free rate), σ is the volatility of the stock price, and dW is a Wiener process (analogous to dz) [?, 1]. This model is commonly referred to as geometric Brownian motion. The first term, $rS dt$, represents the deterministic drift of the stock's price, and the second term, $\sigma S dW$ represents random variance.

3.3 Applying Itô's Lemma to the Option Value

Suppose $V(S, t)$ is the value of a European call option dependant on the price of stock S . Given that the price of S is modled using the Itô process above, Itô's lemma (Equation 5) can be applied, and the result can be rearranged using Equation 6 to obtain:

$$dV = \left(rS \frac{\partial V}{\partial S} + \frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt + \frac{\partial V}{\partial S} \sigma S dW \quad (7)$$

Since the Wiener processes in Equatiion 6 and Equation 7 are the same, a portfolion consisting of positions in both the option and the stock can be created such that the Wiener process is eliminated [1].

3.4 Constructing the Hedged Portfolio

This portfolio as created by Hull, consists of a short position (selling) in one derivative and a long posiiton (buying) in $\frac{\partial V}{\partial S}$ shares of the underlying stock.

$$\Pi = -V + \frac{\partial V}{\partial S} S \quad (8)$$

The change in this portfolio over a smal time interval, dt , can be expressed as:

$$d\Pi = -dV + \frac{\partial V}{\partial S} dS \quad (9)$$

The expressions above for dV (Equation 7) and dS (Equation 6) can be substituted into the equation above to obtain:

$$d\Pi = \left(-\frac{\partial V}{\partial t} - \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt \quad (10)$$

As expected, the Wiener processes cancel, eliminating randomness fromt the expression.

3.5 No Arbitrage Condition

Hull states that because this expression is now free of the randomness induced by the Wiener process, it must be riskless. Therefore, due to the no arbitrage assumption, the portfolio must grow with the risk free rate:

$$dPi = r \Pi dt \quad (11)$$

Substituting Equation 8 and Equation 10 then yields:

$$\left(\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} \right) dt = r \left(V - \frac{\partial V}{\partial S} S \right) dt \quad (12)$$

Then, dividing both sides by dt and rearranging yields the Black-Scholes Merton PDE exactly as seen in Equation 2

$$\frac{\partial V}{\partial t} + \frac{1}{2} \sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (13)$$

4 The Transported PDF Method

4.1 Statistical Description of Turbulent Flows

In *Turbulent Flows*, Stephen B. Pope remarks that “in a turbulent flow, the velocity field $U(x, t)$ is random” [3]. This means that if one were to repeat the same experiment multiple times under the same set of conditions, a given event may, but need not occur, making the event inherently unpredictable. Although a random variable, such as velocity, can not be predicted, the probability of events (e.g., the velocity is less than 10 m/s) can be used to create a Probability Density Function (PDF) that characterizes the random variable. Formally, the probability of any event can be determined by the Cumulative Distribution Function, and the PDF is defined to be the derivative of the CDF. Therefore, according to Pope, a random variable such as the velocity of a turbulent fluid, is completely characterized by its PDF, which can be thought of as the probability per unit distance in the sample space [3].

4.2 Eulerian PDF Transport Equation

The evolution of this Eulerian PDF of velocity in space and time can be represented by a transport equation derived by Pope from the Navier-Stokes equations. The result is the following Eulerian PDF transport equation:

$$\frac{\partial f}{\partial t} + V_i \frac{\partial f}{\partial x_i} = - \frac{\partial}{\partial V_i} \left[f \left\langle \frac{DU_i}{Dt} \middle| \mathbf{V} \right\rangle \right] \quad (14)$$

Where $f(\mathbf{V}; \mathbf{x}, t)$ is the Eulerian PDF of velocity.

4.3 The Generalized Langevin Model

To model the evolution of fluid particle velocities, Pope introduced the generalized Langevin model for a diffusion process. The Langevin equation was originally proposed in 1908 as a stochastic model for the velocity of a microscopic particle undergoing Brownian motion. Pope states that the equation provides a good model for the velocity of a fluid particle in turbulence [3]. The generalized Langevin model used to express the velocity of a fluid particle is expressed below:

$$d\mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*(t), \mathbf{X}^*(t), t) dt + b(\mathbf{X}^*(t), t) d\mathbf{W} \quad (15)$$

where $\mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t)$ is the drift coefficient, $b(\mathbf{X}^*, t)$ is the diffusion coefficient, and $d\mathbf{W}$ is a vector-valued Wiener process [3]. The drift coefficient effectively describes the direction and speed of the system if no randomness were present, and the diffusion coefficient determines the intensity of the random fluctuations induced by the vector-valued Wiener process [3]. According to Pope, the Langevin equation is the simplest stochastic Lagrangian model for a turbulent fluid particle. It should also be noted that the stochastic process $U^*(t)$ is referred to in the text as the Ornstein-Uhlenbeck (OU) process, and although its PDF is already known to evolve by the Fokker-Planck equation [3], Pope still highlights this transition.

4.4 Derivation of the PDF Transport Equation

From the Langevin SDE (Equation 15), Itô calculus can be used to derive the Fokker-Planck equation, which is also known as the forward Kolmogorov equation, and describes the transport of the joint Lagrangian PDF of the particle velocity and position, $f_L^*(\mathbf{V}, \mathbf{x}; t)$. The Fokker-Planck equation can then be divided by the marginal position PDF, $f_X^*(\mathbf{x}, t)$ to obtain an expression for the evolution of the conditional PDF of the particle velocity, $f^*(\mathbf{V}|\mathbf{x}, t)$ [3].

$$\frac{\partial f^*}{\partial t} + V_i \frac{\partial f^*}{\partial x_i} = - \frac{\partial}{\partial V_i} [f^* a_i(\mathbf{V}, \mathbf{x}, t)] + \frac{1}{2} b(\mathbf{x}, t)^2 \frac{\partial^2 f^*}{\partial V_i \partial V_i} \quad (16)$$

Pope notes that the conditional PDF, f^* is the corresponding representation of the Eulerian PDF of velocity, f , for a particle system. Therefore, Equation 16 can be thought of as the Lagrangian equivalent of the Eulerian PDF transport equation, Equation 14. As a result, Equation 16 is commonly referred to as the Lagrangian PDF transport equation, and again, it describes the transport of the conditional PDF of the particle velocity [3].

5 The Connection Between the BSM Equation and the Transported PDF Method

5.1 Structural Equivalence of the SDEs

Both the Black-Scholes-Merton and Lagrangian PDF Transport equations are both derived from stochastic differential equations of the same drift plus diffusion form. Comparing the

geometric Brownian motion SDE from which the BSM equation is derived (Equation 6):

$$dS = rS dt + \sigma S dW$$

with the generalized Langevin SDE (Equation 15) from which the Lagrangian PDF Transport equation is derived:

$$d\mathbf{U}^*(t) = \mathbf{a}(\mathbf{U}^*, \mathbf{X}^*, t) dt + b(\mathbf{X}^*, t) d\mathbf{W}$$

reveals that both equations have the same mathematical structure: deterministic drift term multiplied by dt , plus a diffusion term (random fluctuations) driven by a Wiener process, dW . Furthermore, both equations can be derived using the same mathematical tools, Stochastic Calculus and Itô's lemma.

5.2 Variable Mapping

In short, the structural equivalence between the two mathematical frameworks can be summarized through the direct variable mapping:

Table 1: Variable mapping between Black-Scholes-Merton and PDF Transport frameworks

Quantitative Finance (GBM / BSM)	CFD (Langevin / PDF Transport)
Stock price, S	Particle velocity, $\mathbf{U}^*$
Risk-free rate, r	Drift Coefficient, $\mathbf{a}$
Volatility, σ	Diffusion coefficient, b
Wiener process, dW	Vector-valued Wiener process, $d\mathbf{W}$
Monte Carlo simulation	Lagrangian particle method

However, a few key distinctions should be made. The most prominent is that the BSM equation computes the conditional expectation of a future payoff given the current state, then discounts it back to give the current option price. Whereas, the Lagrangian Transport PDF equation describes how the PDF of velocity evolves over time. In other words, for BSM, the PDF of the stock price is used at expiry (or any future time) to find the expected payoff at that time, then the result is discount it back to its value at a past date. Another difference to highlight is that the drift and diffusion coefficients (r and σ) are constant for the Brownian Motion SDE, while the coefficients in the Langevin SDE ($\mathbf{a}$ and b) depend on the particle's velocity and position at a given time. This difference has a significant impact on the available solution methods and validation techniques, which is discussed in Section 6. Finally, it should be mentioned that the stochastic differential equation for the velocity of a particle uses a vector-valued Wiener process, because the stochastic perturbations act in multiple spatial directions (typically three-dimensional space). Nevertheless, through observing the variable mapping above, a significant connection between not just the two frameworks, but their solution methods as well can be observed.

6 Solution Methods

Both the Black-Scholes-Merton and Lagrangian Transported PDF equations can be solved using Eulerian (grid-based) and Lagrangian (particle-based) methods.

6.1 Eulerian Methods

TBD

6.2 Lagrangian Methods

TBD

6.3 Constant vs. Variable Coefficients

TBD

7 Computational Experiments and Results

MAIN PLOT ON OWN FIG

MC PATHS & PDF TRANSPORT (SIDE BY SIDE)

FRONT & TOP VIEWS OF PDF TRANSPORT (SIDE BY SIDE)

MC ERROR ON OWN FIG

7.1 Problem Description

TBD

7.2 Analytical Solution

TBD

7.3 Finite Difference Solver (Eulerian)

TBD

7.4 Monte Carlo Solver (Lagrangian)

TBD

7.5 Results: Option Pricing

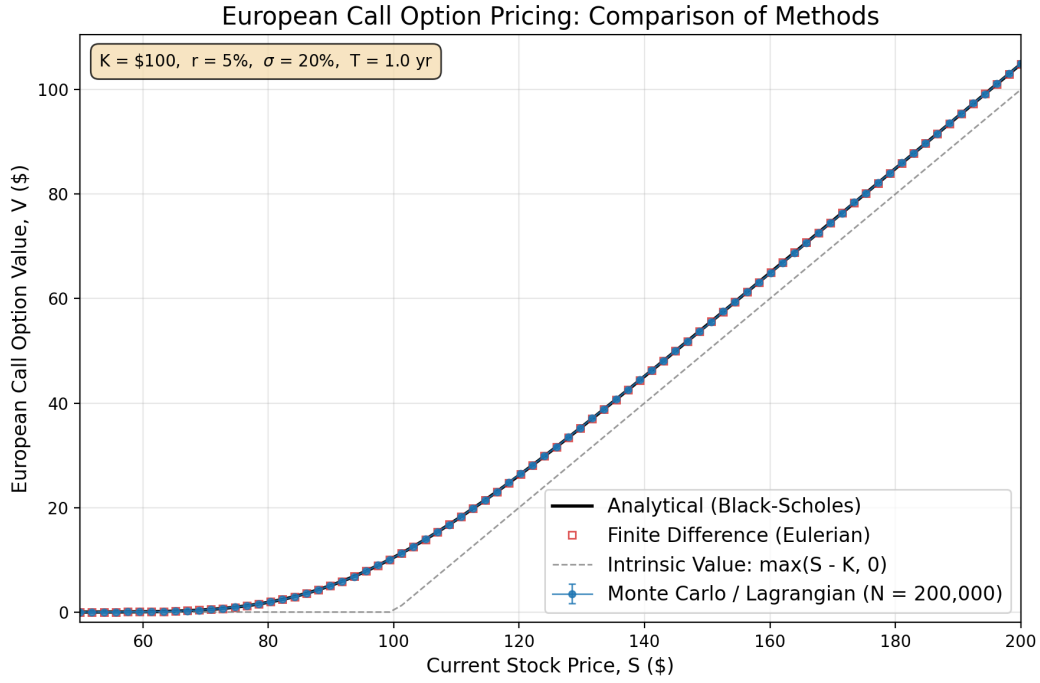


Figure 1: European call option value vs. stock price: comparison of analytical, eulerian, and lagrangian methods

7.6 Results: Monte Carlo Visualization and PDF Transport

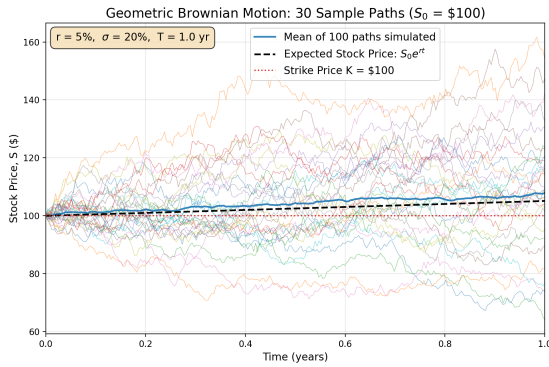


Figure 2: Geometric Brownian Motion: 30 sample paths with mean and expected return.

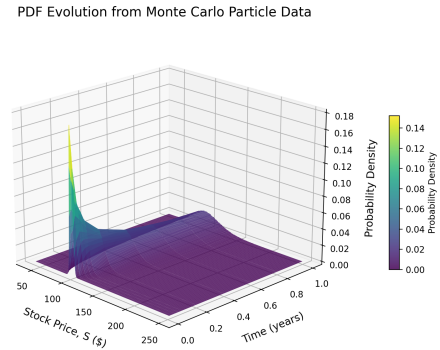


Figure 3: PDF evolution from Monte Carlo particle data

7.7 Results: PDF Transport Continued

TBD

7.8 Results: Monte Carlo Convergence

TBD

8 Conclusions

TBD

References

- [1] J. C. Hull, *Options, Futures, and Other Derivatives*, 10th ed. Boston, MA: Pearson, 2022.
- [2] R. C. Merton, “Theory of rational option pricing,” *Bell Journal of Economics and Management Science*, vol. 4, no. 1, pp. 141–183, 1973.
- [3] S. B. Pope, *Turbulent Flows*. Cambridge: Cambridge University Press, 2000.
- [4] F. Black and M. Scholes, “The pricing of options and corporate liabilities,” *Journal of Political Economy*, vol. 81, no. 3, pp. 637–654, 1973.
- [5] M. Baxter and A. Rennie, *Financial Calculus: An Introduction to Derivative Pricing*. Cambridge: Cambridge University Press, 1996.

A Copy of Program Listing

```
1  """
2  Christian DiPietrantonio
3  ME 5311: Computational Methods to Viscous Flows
4  Term Paper
5  04/19/2026
6  -----
7  Black-Scholes-Merton Solver
8  -----
9
10 This script demonstrates three approaches to pricing a European call option:
11     1. Analytical Black-Scholes formula (closed-form solution)
12     2. Monte Carlo simulation (Lagrangian / stochastic particle method)
13     3. Finite Difference (Eulerian / grid-based PDE solver)
14
15 Methods 2 and 3 are the Lagrangian and Eulerian numerical approaches.
16 """
17
18 import numpy as np
19 from scipy.stats import norm
20 from scipy.linalg import solve_banded
21 import matplotlib.pyplot as plt
22 from matplotlib import cm
23 from mpl_toolkits.mplot3d import Axes3D
24 import os
25
26 # -----
27 # Parameters
28 # -----
29 K      = 100.0 # Strike Price ($)
30 r      = 0.05 # Risk-free interest rate (5%)
31 sigma  = 0.2  # Volatility
32 T      = 1.0  # Time to expiry (1 year)
33
34 # Monte Carlo Settings
35 N_paths = 200000 # Number of simulated paths
36 N_steps = 252    # Time steps (trading days in a year)
37 N_S_pts = 80     # Number of starting prices to evaluate
38
39
40 # Stock price range
41 S_min = 50
42 S_max = 200
43
44 # Output directory
45 out_dir = "figures"
46 os.makedirs(out_dir, exist_ok=True)
47
48 # -----
49 # Analytical Black-Scholes Formula
50 # -----
51 def bs_call(S, K, r, sigma, T):
```

```

52     """
53     Analytical Black-Scholes price for a European call option.
54     Page 644, Black-Scholes
55     Page 91, Baxter-Rennie
56     Page 336, Hull
57
58     Parameters
59     -----
60     S      : current stock price
61     K      : strike price
62     r      : risk free rate
63     sigma  : volatility
64     T      : time to expiration
65
66     Returns
67     -----
68     V      : option prices
69     """
70
71     d1 = (np.log(S / K) + (r + 0.5 * sigma**2) * T) / (sigma * np.sqrt(T))
72     d2 = d1 - sigma * np.sqrt(T)
73     V = S * norm.cdf(d1) - K * np.exp(-r * T) * norm.cdf(d2)
74
75     return V
76
77     # -----
78     # Monte Carlo Solver - Terminal Value Method
79     # -----
80     def mc_option_price(S0, K, r, sigma, T, N_paths):
81         """
82         Price a European call via Monte Carlo using the exact GBM solution. Instead of
            stepping through time, we jump directly to S(T). This works because we only need
            the terminal stock price to compute the payoff. For the PDF evolution, we must use
            the full time-steppind version below.
83
84         Page 217, Hull (Long position European Call option payoff)
85         Page 336, Hull (Option expected value discounted at risk-free rate)
86         Page 471, Hull (Stock price at expiration time)
87         Page 473, Hull (Standard Error of estimate)
88
89
90         Parameters
91         -----
92         S0      : starting stock price
93         K      : strike price
94         r      : risk free rate
95         sigma  : volatility
96         T      : time to expiration
97         N_paths : number of simulated paths
98
99         Returns
100        -----
101         V_mean : estimated option price/value
102         V_std  : standard error of the estimate

```

```

103
104     """
105
106     Z = np.random.standard_normal(N_paths)
107     ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * Z)
108
109     # payoff for a European call
110     payoffs = np.maximum(ST - K, 0.0)
111
112     # discount back to present value
113     V_mean = np.exp(-r * T) * np.mean(payoffs)
114     V_std = np.exp(-r * T) * np.std(payoffs) / np.sqrt(N_paths)
115
116     return V_mean, V_std
117
118 # -----
119 # Monte Carlo Solver - Full Path Simulation
120 # -----
121 def mc_full_paths(S0, r, sigma, T, N_paths, N_steps):
122     """
123     Simulate full GBM paths.
124
125     Page 470, Hull (Discretizing SDEs)
126     Note:
127         - For this example we use the more accurate analytical solution for the GBM SDE.
128           For the transported PDF method, we would discretize the SDE itself.
129
130     Page 471, Hull (Stock price at time step)
131
132     Parameters
133     -----
134     S0      : starting stock price
135     r       : risk free rate
136     sigma   : volatility
137     T       : time to expiration
138     N_paths : number of simulated paths
139     N_steps : number of time steps
140
141     Returns
142     -----
143     S       : all price paths
144     t       : time array
145
146     """
147
148     dt = T / N_steps
149     t = np.linspace(0, T, N_steps + 1)
150
151     S = np.zeros((N_steps + 1, N_paths))
152     S[0, :] = S0
153
154     # generate random increments
155     Z = np.random.standard_normal((N_steps, N_paths))
156
157     # step through time

```

```

156     for i in range(N_steps):
157         S[i+1, :] = S[i, :] * np.exp((r - 0.5 * sigma**2) * dt + sigma * np.sqrt(dt) * Z[i
            , :])
158
159     return S, t
160
161 # -----
162 # Finite Difference Solver (Eulerian / Grid Based PDE Solver)
163 # -----
164 def fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000):
165     """
166     Solver the BSM PDE using implicit finite difference method.
167
168     This is the Eulerian approach: discretize stock price into a grid and solve the PDE
        directly at each point. Important to note that we march backwards in time from t =
        T (expiry) to t=0 (today).
169
170     Parameters
171     -----
172     K      : strike price
173     r      : risk free rate
174     sigma  : volatility
175     T      : time to expiration
176     S_max  : upper bound of stock price grid
177     M      : number of spacial intervals
178     N_t    : number of time steps
179
180     Returns
181     -----
182     S_grid : stock price at grid points
183     V      : option values at t = 0 (today)
184     """
185
186     # ----- Grid Setup -----
187     dS      = S_max / M          # spatial step size
188     dt      = T / N_t           # time step size
189     S_grid = np.linspace(0, S_max, M + 1)
190
191     # ----- Terminal Condition -----
192     # At expiry, option value = payoff = max(S - K, 0)
193     V = np.maximum(S_grid - K, 0.0)
194
195     # ----- Build Tridiagonal Coefficients for Interior Nodes j = 1, ..., M - 1 -----
196     j = np.arange(1, M)
197
198     a_j = 0.5 * dt * (r * j - sigma**2 * j**2)
199     b_j = 1.0 + dt * (sigma**2 * j**2 + r)
200     c_j = -0.5 * dt * (sigma**2 * j**2 + r * j)
201
202     # ----- Format into Banded System -----
203     n_interior = M - 1 # number of unknowns
204     # Banded matrix (l + u + 1, n) where l = 1 lower diagonal, u = 1 upper diagonal
205     # element[row, col] maps to ab[u + row - col, col]
206     ab = np.zeros((3, n_interior))

```

```

207     ab[0, 1:] = c_j[:-1]
208     ab[1,:] = b_j
209     ab[2, :-1] = a_j[1:]
210
211     # ----- Time Marching -----
212     for n in range(N_t):
213         rhs = V[1:M].copy()
214
215         time_to_expiry = (n + 1) * dt
216         V_top = S_max - K * np.exp(-r * time_to_expiry)
217
218         rhs[-1] -= c_j[-1] * V_top
219
220         # Solve the Tridiagonal System
221         V[1:M] = solve_banded((1, 1), ab, rhs)
222
223         # Apply boundary conditions to full solution array
224         V[0] = 0.0
225         V[M] = V_top
226
227     return S_grid, V
228
229     # -----
230     # Figure 1: V(S) Comparison
231     # -----
232     def plot_option_value_curve():
233         """
234         Compare Monte Carlo, Finite Difference, and Analytical Option Prices.
235         """
236
237         print("Computing V(S) curve ...")
238         S_arr = np.linspace(S_min, S_max, N_S_pts)
239
240         #Analytical Solution
241         V_analytical = bs_call(S_arr, K, r, sigma, T)
242
243         # Monte Carlo (loop over starting prices)
244         V_mc = np.zeros(N_S_pts)
245         V_err = np.zeros(N_S_pts)
246
247         for i, S0 in enumerate(S_arr):
248             V_mc[i], V_err[i] = mc_option_price(S0, K, r, sigma, T, N_paths)
249
250         # Finite Difference
251         print("    Running finite difference solver...")
252         S_fd, V_fd = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
253
254         # Interpolate FD results onto same S_arr for comparison
255         V_fd_interp = np.interp(S_arr, S_fd, V_fd)
256
257         # ----- Plot -----
258         fig, ax = plt.subplots(figsize=(9, 6))
259
260         # Analytical Solution

```

```

261 ax.plot(S_arr, V_analytical, 'k-', linewidth = 2, label='Analytical (Black-Scholes)')
262
263 # Finite Difference Solution
264 ax.plot(S_arr, V_fd_interp, 's', color='tab:red', markersize=4, markerfacecolor='none',
265         markeredgewidth=1.0, label='Finite Difference (Eulerian)', alpha=0.8)
266
267 # Monte Carlo (with error bars)
268 ax.errorbar(S_arr, V_mc, yerr=4*V_err, marker='o', color='tab:blue', markersize=4,
269            linewidth=1.0, capsize=2, label=f'Monte Carlo / Lagrangian (N = {N_paths:,})',
270            alpha=0.8)
271
272 # Intrinsic Value for Reference
273 intrinsic = np.maximum(S_arr - K, 0)
274 ax.plot(S_arr, intrinsic, '--', color='tab:gray', linewidth=1.0, alpha=0.8, label='
275         Intrinsic Value: max(S - K, 0)')
276
277 ax.set_xlabel('Current Stock Price, S ($)', fontsize=12)
278 ax.set_ylabel('European Call Option Value, V ($)', fontsize=12)
279 ax.set_title('European Call Option Pricing: Comparison of Methods', fontsize=15)
280 ax.legend(fontsize=12)
281 ax.set_xlim(S_min, S_max)
282 ax.set_ylim(bottom=-2)
283 ax.grid(True, alpha=0.3)
284
285 param_text = (fr'K = \${K:.0f}, r = {r:.0%}, $\sigma$ = {\sigma:.0%}, T = {T:.1f} yr')
286 ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=10,
287        verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor=
288        'black', pad=0.5, alpha=0.8))
289
290 fig.tight_layout()
291 fig.savefig(f'{out_dir}/figure01_option_value_curve.png', dpi=200)
292 plt.close(fig)
293 print(f'    Saved: {out_dir}/figure01_option_value_curve.png')
294
295 return S_arr, V_analytical, V_mc, V_err, V_fd_interp
296
297 # -----
298 # Figure 2: Sample GBM Paths
299 # -----
300
301 def plot_sample_paths(S0=100.0, N_show=30, N_sim=100):
302     """
303     Show a handful of GBM trajectories to illustrate stochastic behavior.
304
305     Page 323, 325, Hull (Expected Return)
306     """
307     print("Simulating sample paths...")
308     S, t = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
309
310     fig, ax = plt.subplots(figsize=(9, 6))
311
312     # plot subset of paths
313     for j in range(N_show):
314         ax.plot(t, S[:, j], linewidth=0.5, alpha=0.6)

```

```

309
310 # overlay the mean of the simulated paths
311 S_mean_sim = np.mean(S, axis=1)
312 ax.plot(t, S_mean_sim, color='tab:blue', linewidth=2.0, alpha=0.9, label=f'Mean of {
    N_sim} paths simulated')
313
314 # overlay the expected return
315 S_expected = S0 * np.exp(r * t)
316 ax.plot(t, S_expected, 'k--', linewidth=2.0, label=f'Expected Stock Price:  $S_0 e^{\{
    rt\}}$ ')
317
318 ax.axhline(K, color='tab:red', linestyle=':', linewidth=1.5, label=f'Strike Price K =
     $\{K:.0f\}$ ')
319
320 ax.set_xlabel('Time (years)', fontsize=12)
321 ax.set_ylabel('Stock Price, S ($)', fontsize=12)
322 ax.set_title(fr'Geometric Brownian Motion: {N_show} Sample Paths ( $S_0 = \{S0:.0f\}$ )
    ', fontsize=15)
323 ax.legend(fontsize=12, loc='upper center', bbox_to_anchor=(0.6, 1.0))
324 ax.set_xlim(0, T)
325 ax.grid(True, alpha=0.3)
326
327 param_text = (fr'r = {r:.0%},  $\sigma = \{sigma:.0\}$ , T = {T:.1f} yr')
328 ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12,
    verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor=
    'black', pad=0.5, alpha=0.8))
329
330 fig.tight_layout()
331 fig.savefig(f'{out_dir}/figure02_GMB_sample_paths.png', dpi=200)
332 plt.close(fig)
333 print(f'    Saved: {out_dir}/figure02_GMB_sample_paths.png')
334
335 return S, t
336
337 # -----
338 # Figure 3: PDF Evolution - 3D Surface
339 # -----
340
341 def plot_pdf_evolution(S0=100.0, N_sim=200000):
342     """
343     Visualize how the probability density of a stock price evolves over time using Monte
344     Carlo particle data.
345     Note:
346     - The exact GBM sol could be used to construct the PDF using the PDF function for
347       a lognormal distribution. However, here the PDF is constructed using MC data
348       , because no PDF function is available for the Langevin SDE.
349
350     Page 322, Hull (Mean and Standard Deviation of  $\ln ST$ )
351     Page 84, Montgomery, Runger, Hubele
352     """
353     print("Computing PDF evolution surface from Monte Carlo data...")
354
355     t_snapshots = np.array([0.01, 0.025, 0.05, 0.075, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0])

```

```

354 S_bins = np.linspace(40, 250, 500)
355 S_bin_centers = 0.5 * (S_bins[:-1] + S_bins[1:])
356
357 # Simulate paths
358 S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
359
360 # Build PDF at each time snapshot
361 PDF_mc = np.zeros((len(t_snapshots), len(S_bin_centers)))
362 for i, t_val in enumerate(t_snapshots):
363     idx = np.argmin(np.abs(t_all - t_val))
364     counts, _ = np.histogram(S_all[idx, :], bins=S_bins, density=True)
365     PDF_mc[i, :] = counts
366
367 fig = plt.figure(figsize=(9, 6))
368 ax = fig.add_subplot(111, projection='3d')
369 T_mesh, S_mesh = np.meshgrid(t_snapshots, S_bin_centers, indexing='ij')
370 surf = ax.plot_surface(S_mesh, T_mesh, PDF_mc, cmap='viridis', alpha=0.85, rstride=1,
371                        cstride=3, edgecolor='none')
372
373 ax.set_xlabel('Stock Price, S ($)', fontsize=12, labelpad=10)
374 ax.set_ylabel('Time (years)', fontsize=12, labelpad=10)
375 ax.set_zlabel('Probability Density', fontsize=12, labelpad=10)
376 ax.set_title('PDF Evolution from Monte Carlo Particle Data', fontsize=15)
377 ax.view_init(elev=20, azimuth=-45)
378 fig.colorbar(surf, ax=ax, shrink=0.45, aspect=15, label='Probability Density', pad
379              =0.10)
380
381 fig.tight_layout()
382 fig.savefig(f'{out_dir}/figure03_3d_pdf_evolution.png', dpi=200)
383 plt.close(fig)
384 print(f'    Saved: {out_dir}/figure03_3d_pdf_evolution.png')
385 return t_snapshots, S_bin_centers, PDF_mc
386
387 # -----
388 # Figure 4: PDF Evolution - 2D Slices
389 # -----
390 def plot_pdf_slices(S0=100.0, N_sim=200000):
391     """
392     Monte Carlo histograms overlaid with analytical log-normal curves. This demonstrates
393     that the Lagrangian statistics converge to the Eulerian PDF.
394
395     Page 322, Hull (Mean and Standard Deviation of ln ST)
396     Page 84, Montgomery, Runger, Hubele
397     """
398     print("Computing PDF time slices...")
399
400     t_slices = [0.02, 0.1, 0.25, 0.5, 1.0]
401     S_grid = np.linspace(40, 250, 500)
402
403     # Simulate paths
404     S_all, t_all = mc_full_paths(S0, r, sigma, T, N_sim, N_steps)
405
406     colors = cm.plasma(np.linspace(0.1, 0.9, len(t_slices)))

```

```

405 fig, ax = plt.subplots(figsize=(9, 6))
406 pdf_max = 0.0
407
408
409 for i, t_val in enumerate(t_slices):
410     # Monte Carlo Histogram
411     idx = np.argmin(np.abs(t_all - t_val))
412     ax.hist(S_all[idx, :], bins=120, range=(40, 250), density=True, alpha=0.5, color=
         colors[i], edgecolor='none')
413
414     # Analytical log-normal PDF
415     mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val
416     sigma_ln = sigma * np.sqrt(t_val)
417     pdf_vals = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi))) * np.exp(-0.5 * ((np.log(
         S_grid) - mu_ln) / sigma_ln)**2))
418     pdf_max = max(pdf_max, np.max(pdf_vals))
419     ax.plot(S_grid, pdf_vals, color=colors[i], linewidth=2.0, label=f't = {t_val:.2f}
         years')
420
421     ax.axvline(S0, color='black', linestyle=':', linewidth=2.0, alpha=0.5, label=fr'$S_0$
         = \${S0:.0f}$')
422
423     ax.set_xlabel('Stock Price, S ($)', fontsize=12)
424     ax.set_ylabel('Probability Density', fontsize=12)
425     ax.set_title('PDF Evolution: Monte Carlo Histograms vs. Analytical', fontsize=15)
426     ax.legend(fontsize=12, loc='upper right')
427     ax.set_xlim(40, 250)
428     ax.set_ylim(0, pdf_max * 1.1)
429     ax.grid(True, alpha=0.3)
430
431     param_text = (fr'$S_0$ = \${S0:.0f}, r = {r:.0%}, $\sigma$ = {sigma:.0%}, N = {N_sim
         :}, paths')
432     ax.text(0.02, 0.97, param_text, transform=ax.transAxes, fontsize=12,
         verticalalignment='top', bbox=dict(boxstyle='round', facecolor='wheat', edgecolor=
         'black', pad=0.5, alpha=0.8))
433
434     fig.tight_layout()
435     fig.savefig(f'{out_dir}/figure04_2D_pdf_slices.png', dpi=200)
436     plt.close(fig)
437     print(f'    Saved: {out_dir}/figure04_2D_pdf_slices.png')
438
439 # -----
440 # Figure 5: Monte Carlo Convergence
441 # -----
442 def plot_convergence(S0=100.0):
443     """
444     Show how the MC estimate converges as more paths are simulated.
445
446     Page 473, Hull (MC Standard Error/Confidence intervals)
447     """
448
449     print("Computing convergence study...")
450     V_exact = bs_call(S0, K, r, sigma, T)
451     N_range = np.logspace(2, 6, 30).astype(int)

```

```

452 V_estimates = np.zeros(len(N_range))
453 V_errors    = np.zeros(len(N_range))
454
455 np.random.seed(11)
456 for i, N in enumerate(N_range):
457     V_estimates[i], V_errors[i] = mc_option_price(S0, K, r, sigma, T, N)
458
459 fig, ax = plt.subplots(figsize=(9, 6))
460 ax.semilogx(N_range, V_estimates, 'o-', color='tab:blue', markersize=5, linewidth
    =1.0, label='MC Estimate')
461 ax.fill_between(N_range, V_estimates - 2*V_errors, V_estimates + 2*V_errors, alpha
    =0.2, color='tab:blue', label=r'$\pm 2 \sigma$ (95% CI)')
462 ax.axhline(V_exact, color='tab:red', linestyle='--', linewidth=2.0, label=f'Exact (BS
    ) = ${V_exact:.2f}$')
463
464 ax.set_xlabel('Number of Monte Carlo Paths, N', fontsize=12)
465 ax.set_ylabel('Estimated Option Value, V ($)', fontsize=12)
466 ax.set_title(fr'Monte Carlo Convergence ($S_0$ = ${S0:.0f}$)', fontsize=15)
467 ax.legend(fontsize=12)
468 ax.grid(True, alpha=0.3)
469
470 fig.tight_layout()
471 fig.savefig(f'{out_dir}/figure05_MC_convergence.png', dpi=200)
472 plt.close(fig)
473 print(f'    Saved: {out_dir}/figure05_MC_convergence.png')
474
475 # -----
476 # Figure 6: PDF Heatmap
477 # -----
478 def plot_pdf_heatmap(S0=100.0):
479     """
480     2D Heatmap: x = stock price, y = time, color = probability density.
481
482     Page 84, Montgomery, Runger, Hubele (Mean and Varince in Price Space)
483     """
484
485     print("Computing PDF heatmap...")
486     t_grid = np.linspace(0.05, T, 500)
487     S_grid = np.linspace(40, 250, 500)
488
489     PDF = np.zeros((len(t_grid), len(S_grid)))
490     for i, t_val in enumerate(t_grid):
491         mu_ln = np.log(S0) + (r - 0.5 * sigma**2) * t_val
492         sigma_ln = sigma * np.sqrt(t_val)
493         PDF[i, :] = (1.0 / (S_grid * sigma_ln * np.sqrt(2*np.pi))) * np.exp(-0.5 * ((np.
            log(S_grid) - mu_ln) / sigma_ln)**2))
494
495     fig, ax = plt.subplots(figsize=(9, 6))
496     S_mesh, t_mesh = np.meshgrid(S_grid, t_grid)
497     pcm = ax.pcolormesh(S_mesh, t_mesh, PDF, cmap='inferno', shading='gouraud')
498
499     S_expected = S0 * np.exp(r * t_grid)
500     ax.plot(S_expected, t_grid, 'w--', linewidth=2.0, label=r'$E(S_T) = S_0 e^{\rt}$')
501

```

```

502     for sign, lbl in [(1, r'+1$\sigma$'), (-1, r'-1$\sigma$')]:
503         mu_ln      = np.log(S0) + (r - 0.5 * sigma**2) * t_grid
504         sigma_ln   = sigma * np.sqrt(t_grid)
505         S_bound    = np.exp(mu_ln + sign * sigma_ln)
506         ax.plot(S_bound, t_grid, 'w:', linewidth=1.0, label=lbl, alpha=0.7)
507
508     ax.set_xlabel('Stock Price, S ($)', fontsize=12)
509     ax.set_ylabel('Time (years)', fontsize=12)
510     ax.set_title('PDF Evolution Heatmap\n' \
511                 fr'$S_0$ = ${S0:.0f}, r = {r:.0%}, $\sigma$ = ${sigma:.0%})', fontsize
512                 =15)
513     ax.legend(fontsize=12, loc='upper right', facecolor='black', edgecolor='white',
514              labelcolor='white')
515     fig.colorbar(pcm, ax=ax, label = 'Probability Density')
516     ax.set_xlim(40, 250)
517
518     fig.tight_layout()
519     fig.savefig(f'{out_dir}/figure06_pdf_heatmap.png', dpi=200)
520     plt.close(fig)
521     print(f'    Saved: {out_dir}/figure06_pdf_heatmap.png')
522
523 # -----
524 # Main Driver
525 # -----
526 def main():
527     print("=" * 120)
528     print("BSM Solver - Monte Carlo + Finite Difference + Analytical")
529     print("=" * 120)
530     print(fr'Parameters: K=${K}, r={r:.2%}, $\sigma$={sigma:.2%}, T={T} years')
531     print(f'Monte Carlo: {N_paths:,} paths, {N_steps} time steps')
532     print("=" * 120)
533
534     np.random.seed(2026)
535
536     S_arr, V_an, V_mc, V_err, F_fd = plot_option_value_curve()
537     S_paths, t_arr                = plot_sample_paths()
538     t_snap, S_centers, PDF_mc    = plot_pdf_evolution()
539     plot_pdf_slices()
540     plot_convergence()
541     plot_pdf_heatmap()
542
543     print("\n" + "=" * 120)
544     print("V(S) at selected stock prices")
545     print("=" * 120)
546     print(f'{'S ($)':>8} {'Analytical':>12} {'Monte Carlo':>12} {'Finite Difference':>12}
547           {'MC Error':>8}')
548     print("-" * 120)
549
550     S_fd_check, V_fd_check = fd_bsm(K, r, sigma, T, S_max=500.0, M=500, N_t=5000)
551     check_prices = [60, 80, 100, 120, 140, 160]
552     for S0 in check_prices:
553         V_an_val      = bs_call(S0, K, r, sigma, T)
554         V_mc_val, V_mc_err = mc_option_price(S0, K, r, sigma, T, N_paths)
555         V_fd_val      = np.interp(S0, S_fd_check, V_fd_check)

```

```
553         print(f'{S0:>8.0f} {V_an_val:>12.2f} {V_mc_val:>12.2f} {V_fd_vall:>12.2f} {  
554             V_mc_err:>12.2f}')  
555     print('\nAll figures saved to:', out_dir)  
556     print('Done!')  
557  
558 if __name__ == "__main__":  
559     main()
```